

Forecasting Electricity Demand

A worked case study in time-series forecasting, with a real generation-mix leakage trap

Your Name

2026-09-30

Table of contents

1	Introduction	2
2	Data	2
2.1	Two real data-quality issues — and they are not the same issue	2
2.2	Exploring the series and its generation mix	3
3	Statistical rigor before modeling	6
3.1	Is this series stationary?	6
3.2	Autocorrelation, used to justify lag choices empirically	6
4	Feature engineering	7
4.1	The leakage trap in the generation-mix columns	7
4.2	A leakage-safe way to reuse a seasonal-decomposition idea	8
5	Methodology	9
5.1	Why the train/test split is chronological, not shuffled	9
5.2	A naive baseline, measured before trusting any model	9
5.3	Model comparison, via TimeSeriesSplit	9
6	Final chronological holdout	11
7	Model interpretability	11
8	Discussion: what this project can't do, and what a Vietnam version would need	13
9	Conclusion	13

1 Introduction

I built this project to predict hourly electricity demand (**Consumption**, megawatts) for Romania's national grid, using real, published data from 2019 onward, together with its generation mix (nuclear, wind, hydro, oil & gas, coal, solar, biomass). I picked demand forecasting as a deliberately different kind of project for this portfolio: every other project treats each row as independent, while this one is a **time series**, where the order of the rows *is* the problem and a shuffled train/test split would quietly leak.

I originally built this project on US grid data (PJM Interconnection), then switched to this Romanian dataset specifically because the generation-mix breakdown is genuinely useful, interesting data the US version didn't have — it lets me ask whether recent generation mix carries any forecasting signal, and it comes with its own real energy-transition story.

One thing worth flagging upfront: **this dataset is live and continuously updated** — checked directly, an earlier download had 46,002 rows through April 2024, the version used here has 62,810 rows through March 2026, with older rows unchanged. Every number below reflects one snapshot, not a fixed, versioned file.

I still couldn't find a real, public, hourly demand dataset for Vietnam's own grid. Romania is a different climate and market structure, but grid-scale demand forecasting is the same *kind* of problem regardless of country, and Romania's growing renewable share raises questions increasingly relevant to Vietnam's own energy transition too.

2 Data

```
from energy_demand_forecasting import config, data

df = data.load_raw()
print(f"{len(df)} rows, {df[config.DATETIME_COL].min()} to {df[config.DATETIME_COL].max()}")
```

62810 rows, 2019-01-01 00:00:00 to 2026-03-14 23:00:00

2.1 Two real data-quality issues — and they are not the same issue

```
dupes = df[df[config.DATETIME_COL].duplicated(keep=False)]
full_range = pd.date_range(df[config.DATETIME_COL].min(), df[config.DATETIME_COL].max(), freq="H")
missing = full_range.difference(df[config.DATETIME_COL])
print(f"{len(dupes)} duplicate timestamps, {len(missing)} missing timestamps")
```

```
gaps = data.gap_lengths(df)
print(f"{len(gaps)} separate gaps, longest: {gaps.max()} hours")
```

```
18 duplicate timestamps, 321 missing timestamps
78 separate gaps, longest: 83 hours
```

Every duplicate falls on an October date at **3am** local time — the same Daylight Saving Time fall-back pattern as any DST-observing country, just an hour later on the clock than the US version of this project found, since Romania is in the Eastern European time zone. 2019-2023 show exactly 1 missing hour per year (the spring-forward gap). **2024-2025 are a completely different story**: a single gap of **83 consecutive missing hours**, consistent with a real reporting/scraping outage in whatever live feed this dataset’s maintainer pulls from — not a calendar quirk. I fixed both the same way (average duplicates, linearly interpolate gaps against a strictly regular hourly index), but want to be upfront that interpolating an 83-hour gap is a much rougher approximation than the isolated 1-hour DST gaps — there’s no way to recover 83 hours of real demand swings from a straight line between two endpoints.

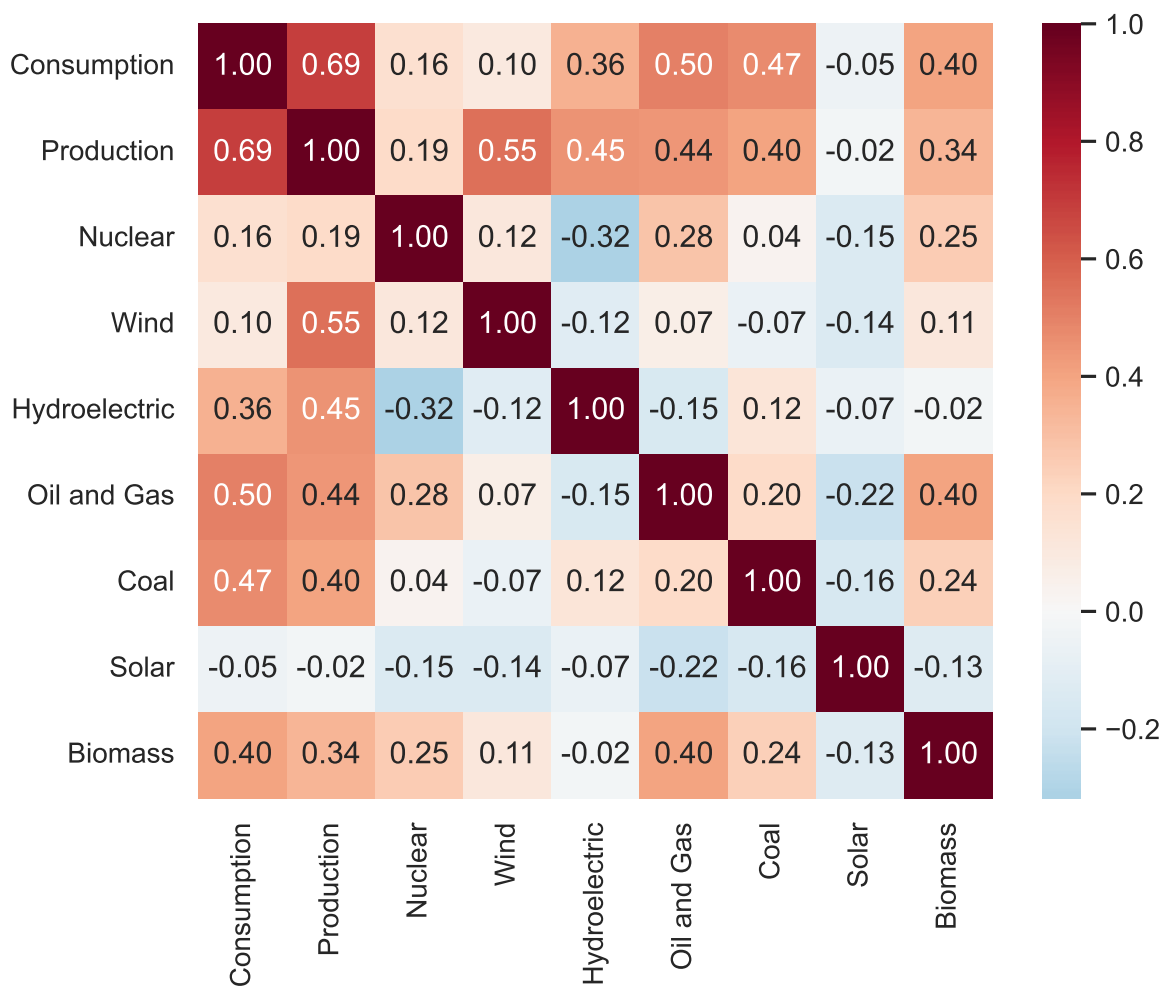
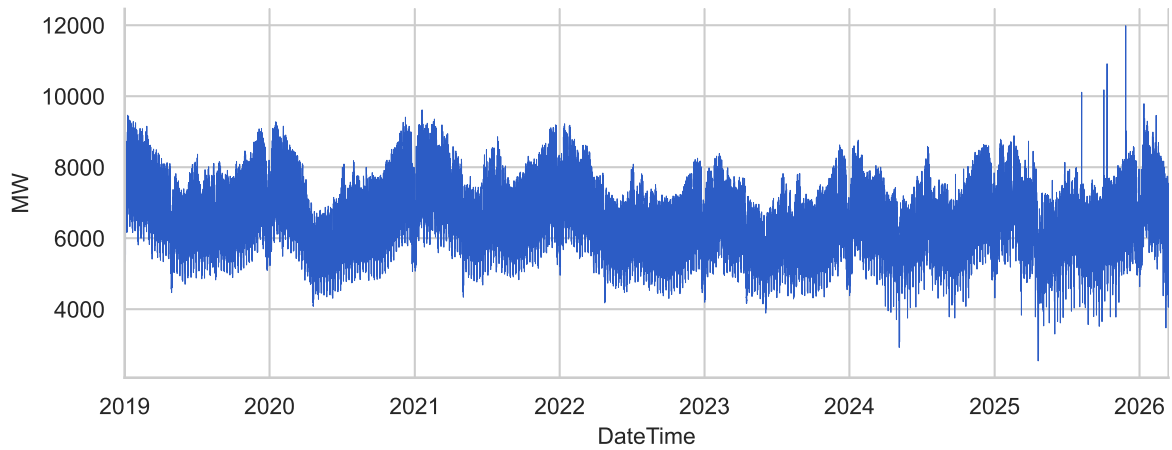
```
frame = data.load_clean_frame()
print(f"{len(frame)} hours, strictly regular: {(frame.index.to_series().diff().dropna() == p
```

```
63120 hours, strictly regular: True
```

2.2 Exploring the series and its generation mix

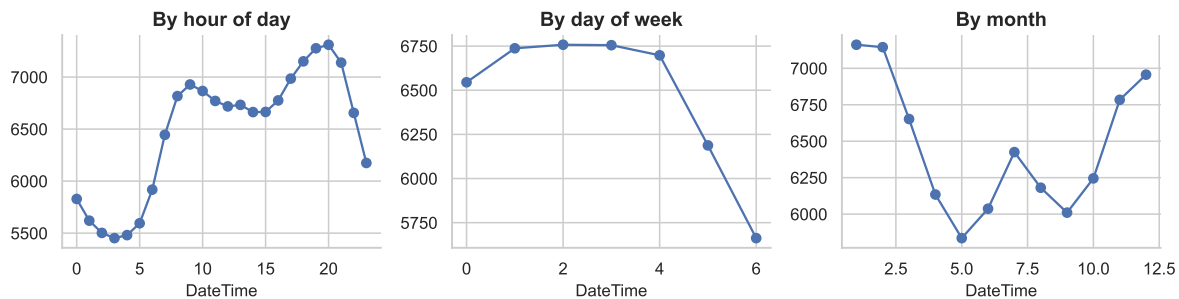
```
fig, ax = plt.subplots(figsize=(9, 3.2))
frame[config.TARGET_COL].plot(ax=ax, linewidth=0.4, color=PALETTE[0])
ax.set_ylabel("MW")
plt.show()

corr = frame.corr()
fig, ax = plt.subplots(figsize=(7, 5.5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0, ax=ax)
plt.show()
```



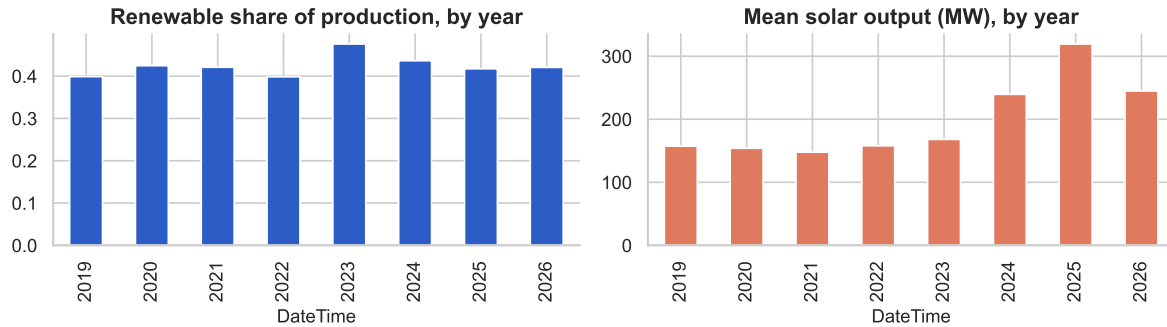
Consumption correlates most with Production (0.69, as expected) and the two dispatchable fossil sources, Oil and Gas (0.50) and Coal (0.47) — the grid operator leans on fossil generation specifically to follow demand swings. Solar is essentially uncorrelated with demand (-0.05), which is exactly what I'd expect: solar output follows daylight and weather, not consumption.

```
fig, axes = plt.subplots(1, 3, figsize=(11, 3.0))
frame.groupby(frame.index.hour)[config.TARGET_COL].mean().plot(ax=axes[0], marker="o", title="By hour of day")
frame.groupby(frame.index.dayofweek)[config.TARGET_COL].mean().plot(ax=axes[1], marker="o", title="By day of week")
frame.groupby(frame.index.month)[config.TARGET_COL].mean().plot(ax=axes[2], marker="o", title="By month")
plt.tight_layout()
plt.show()
```



A single evening peak (~8-9pm, ~7,300 MW) and overnight trough (~3-4am, ~5,450 MW) — a cleaner single-peak daily shape than this project's original US version, which had a secondary morning bump. The monthly pattern is **winter-dominant, not bimodal**: demand peaks in January/February and troughs in May, with much less of a summer cooling bump than the US version showed — consistent with Romania's historically lower AC penetration.

```
renewable_share = frame[config.RENEWABLE_COLS].sum(axis=1) / frame["Production"]
fig, axes = plt.subplots(1, 2, figsize=(10, 3.0))
renewable_share.groupby(frame.index.year).mean().plot.bar(ax=axes[0], title="Renewable share")
frame.groupby(frame.index.year)["Solar"].mean().plot.bar(ax=axes[1], title="Mean solar output")
plt.tight_layout()
plt.show()
```



The clearest “genuinely useful and interesting” finding from having the generation-mix breakdown at all: renewable share has held around 40-48% of production every year since 2019, but **mean solar output roughly doubled between 2023 (~168 MW) and 2025 (~319 MW)** — a real, checkable signature of Romania’s recent solar buildout.

3 Statistical rigor before modeling

3.1 Is this series stationary?

```
from energy_demand_forecasting import model

adf_result = model.stationarity_test(frame[config.TARGET_COL])
print(f"ADF statistic: {adf_result['statistic']:.2f}, p-value: {adf_result['p_value']:.4g}")
```

ADF statistic: -14.38, p-value: 9.086e-27

Formally stationary (no unit root) — which looked wrong to me at first given the obvious daily/monthly patterns above. The resolution: ADF specifically tests for a *stochastic trend*, not for seasonality. A strong, bounded, repeating pattern always reverts toward its typical shape rather than wandering further from it, so it doesn’t have a unit root even though it clearly isn’t “flat.”

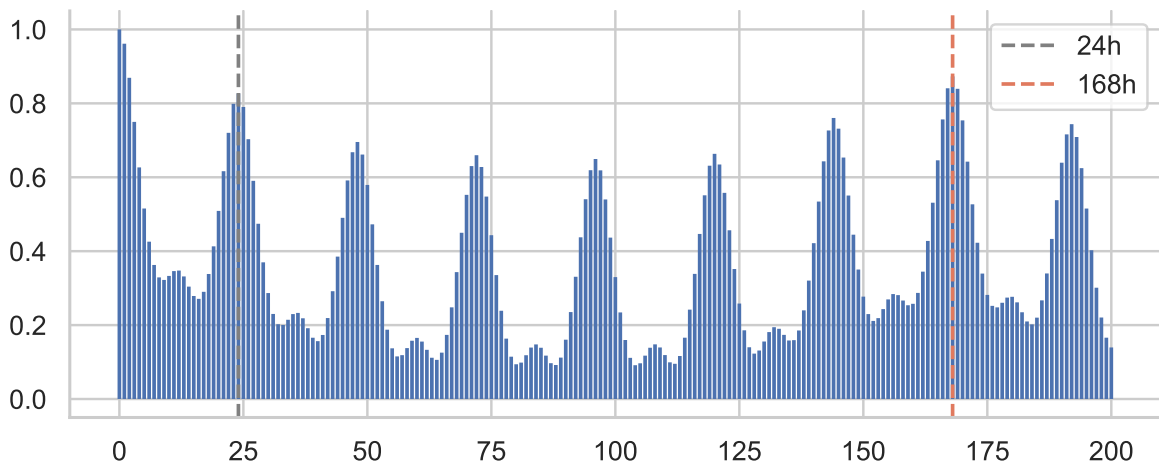
3.2 Autocorrelation, used to justify lag choices empirically

```
acf_values = model.autocorrelation(frame[config.TARGET_COL], n_lags=200)
fig, ax = plt.subplots(figsize=(8, 3.0))
ax.stem(range(len(acf_values)), acf_values, markerfmt=" ", basefmt=" ")
```

```

ax.axvline(24, linestyle="--", color="gray", label="24h")
ax.axvline(168, linestyle="--", color=PALETTE[3], label="168h")
ax.legend()
plt.show()
for lag in [1, 24, 48, 168]:
    print(f"ACF at lag {lag}h: {acf_values[lag]:.4f}")

```



```

ACF at lag 1h: 0.9614
ACF at lag 24h: 0.8267
ACF at lag 48h: 0.6955
ACF at lag 168h: 0.8730

```

Autocorrelation is genuinely high at both 24h (0.83) and 168h (0.87 — if anything slightly stronger), with a noticeably weaker mid-point at 48h (0.70) — empirical justification for these two specific lags, rather than picking them by convention.

4 Feature engineering

4.1 The leakage trap in the generation-mix columns

Production and its 7 source breakdowns are recorded at the **same hour** as Consumption, not before it. Using them directly would be circular for a real forecasting task, since generation is dispatched in response to demand, not fixed in advance. Every generation-mix feature is lagged by 24 hours.

```
from energy_demand_forecasting import features
```

```
seasonal_lookup = features.fit_seasonal_lookup(frame[config.TARGET_COL].iloc[: int(len(frame)
X, y = features.split_features_target(frame, seasonal_lookup=seasonal_lookup)
X[["lag_24h", "lag_168h", "Solar_lag_24h", "net_export_lag_24h", "renewable_share_lag_24h"]]
```

	lag_24h	lag_168h	Solar_lag_24h	net_export_lag_24h	renewable_share_lag_24h
DateTime					
2026-03-14 21:00:00	7041.0	6402.0	0.0	-185.0	0.441219
2026-03-14 22:00:00	6426.0	6234.0	0.0	115.0	0.415839
2026-03-14 23:00:00	6019.0	5777.0	0.0	348.0	0.394220

4.2 A leakage-safe way to reuse a seasonal-decomposition idea

A natural next step is decomposing the series into trend/seasonal/ residual components and adding them as features. Doing this the most direct way — fitting `statsmodels`' decomposition on the whole series before splitting — leaks: the trend component is a **centered** moving average using points both before *and after* each timestamp, so fitting it on the full series means training features are quietly built with future test-period information. `fit_seasonal_lookup` fits the decomposition **only on the training split** and reuses just the strictly periodic seasonal component (24 fixed hour-of-day offsets, safe to reapply to any timestamp without recomputation).

```
X_train, X_test, y_train, y_test = model.chronological_train_test_split(X, y)
pipe_with = model.build_pipeline(model.lightgbm_estimator())
pipe_with.fit(X_train, y_train)
scores_with = model.regression_scores(y_test, pipe_with.predict(X_test))

X_without, y_without = features.split_features_target(frame)
X_train_wo, X_test_wo, y_train_wo, y_test_wo = model.chronological_train_test_split(X_without, y_without)
pipe_without = model.build_pipeline(model.lightgbm_estimator())
pipe_without.fit(X_train_wo, y_train_wo)
scores_without = model.regression_scores(y_test_wo, pipe_without.predict(X_test_wo))

print("WITH seasonal_24h: ", {k: round(v, 4) for k, v in scores_with.items()})
print("WITHOUT seasonal_24h:", {k: round(v, 4) for k, v in scores_without.items()})
```

```
WITH seasonal_24h: {'mae': 259.0764, 'rmse': 423.3834, 'mape': 0.0428}
WITHOUT seasonal_24h: {'mae': 258.6064, 'rmse': 424.0605, 'mape': 0.0428}
```

An honest negative result: adding the seasonal-decomposition feature makes essentially no difference (identical MAPE to 4 decimal places). LightGBM’s tree splits likely already extract everything useful from the cyclical `hour_sin/hour_cos` encoding plus the lag/rolling features — worth reporting the idea that didn’t help, not only the ones that did.

5 Methodology

5.1 Why the train/test split is chronological, not shuffled

Lag and rolling-window features are built from each row’s own recent past, so a random shuffle would leak near-future information into training rows sitting next to shuffled-in test rows. The split is by position along the time-sorted index (last 15% of hours as test), and cross-validation uses `TimeSeriesSplit`, never `KFold`.

5.2 A naive baseline, measured before trusting any model

```
baseline_scores = model.naive_seasonal_baseline(X_test_wo, y_test_wo)
print({k: round(v, 4) for k, v in baseline_scores.items()})
```

```
{'mae': 469.9865, 'rmse': 743.9772, 'mape': 0.078}
```

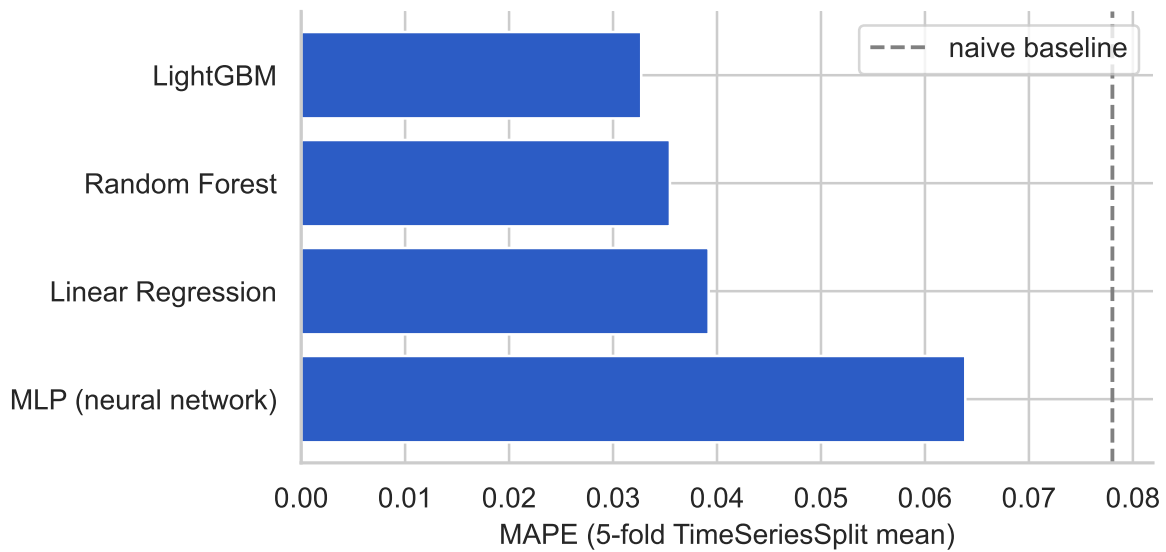
“Same hour last week” is off by ~**7.8%** on average (MAPE) — note this test period includes some of the messy, interpolated 2024-2025 outage stretches, so this baseline is measured under harder conditions than 2019-2023 alone would give.

5.3 Model comparison, via `TimeSeriesSplit`

```
cv_results = []
for name, factory in model.MODEL_FACTORIES.items():
    scores = model.cross_validate_pipeline(X_without, y_without, estimator=factory(), cv=5)
    cv_results.append({"model": name, **scores})
cv_df = pd.DataFrame(cv_results).sort_values("mape_mean")
cv_df[["model", "mae_mean", "rmse_mean", "mape_mean", "mape_std"]]
```

	model	mae_mean	rmse_mean	mape_mean	mape_std
2	LightGBM	201.451715	296.578291	0.032711	0.007269
1	Random Forest	217.293322	331.002114	0.035472	0.008034
0	Linear Regression	246.806452	348.224245	0.039196	0.007359
3	MLP (neural network)	409.488313	533.091900	0.063881	0.052476

```
fig, ax = plt.subplots(figsize=(6, 3.2))
ax.barh(cv_df["model"], cv_df["mape_mean"], color=PALETTE[0])
ax.axvline(baseline_scores["mape"], linestyle="--", color="gray", label="naive baseline")
ax.set_xlabel("MAPE (5-fold TimeSeriesSplit mean)")
ax.legend()
plt.gca().invert_yaxis()
plt.show()
```



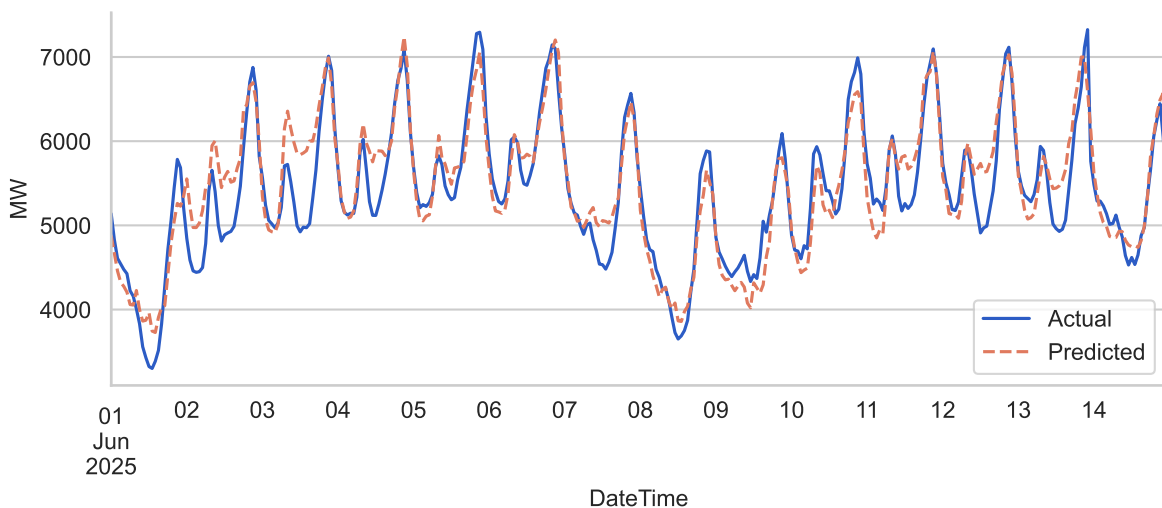
Every model clears the naive baseline by a wide margin, and **LightGBM wins clearly** — including against MLP (neural network), a lightweight feedforward net (`sklearn.neural_network.MLPRegressor`, zero new dependencies) included specifically to test whether a neural approach earns its extra complexity here. It lands close to Random Forest, but neither catches LightGBM — a useful, concrete data point against reaching for a neural network on a tabular, feature-engineered version of this kind of problem.

6 Final chronological holdout

```
final_pipeline = model.train_pipeline(X_train_wo, y_train_wo, estimator=model.lightgbm_estimator)
test_preds = final_pipeline.predict(X_test_wo)
print({k: round(v, 4) for k, v in model.regression_scores(y_test_wo, test_preds).items()})
```

```
{'mae': 258.6064, 'rmse': 424.0605, 'mape': 0.0428}
```

```
fig, ax = plt.subplots(figsize=(9, 3.2))
sample_start, sample_end = "2025-06-01", "2025-06-14"
y_test_wo.loc[sample_start:sample_end].plot(ax=ax, label="Actual", color=PALETTE[0])
pd.Series(test_preds, index=y_test_wo.index).loc[sample_start:sample_end].plot(ax=ax, label="Predicted", color=PALETTE[1])
ax.set_ylabel("MW")
ax.legend()
plt.show()
```

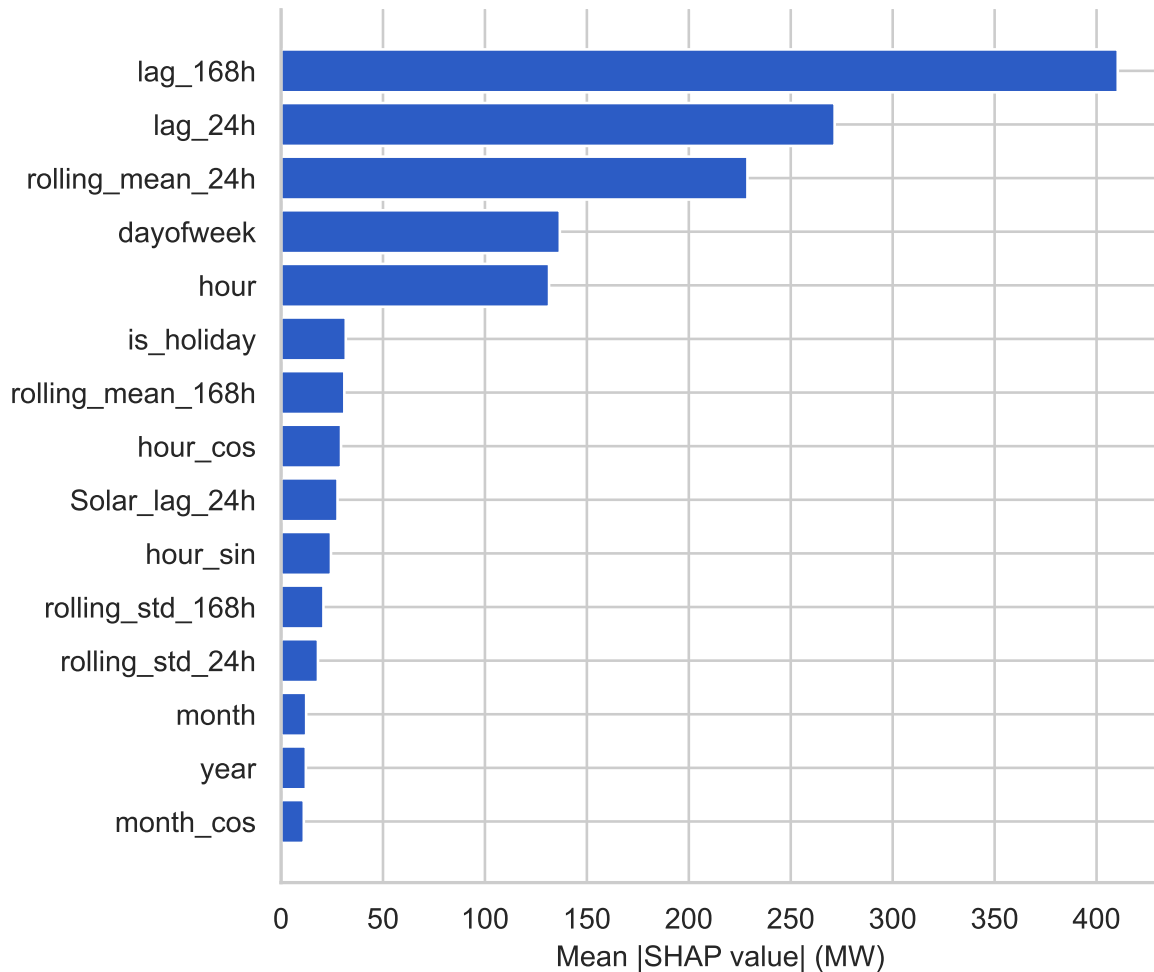


7 Model interpretability

```
from energy_demand_forecasting.interpretability import compute_shap_values, top_shap_features

explanation, X_transformed = compute_shap_values(final_pipeline, X_without, max_samples=500)
top = top_shap_features(explanation, top_n=15).sort_values("mean_abs_shap")
```

```
fig, ax = plt.subplots(figsize=(6, 6))
ax.barh(top["feature"], top["mean_abs_shap"], color=PALETTE[0])
ax.set_xlabel("Mean |SHAP value| (MW)")
plt.show()
```



lag_168h outranks lag_24h here, matching the ACF finding above where the weekly echo was slightly stronger. The genuine surprise: **Solar_lag_24h lands in the top 10**, ahead of is_holiday. I didn't expect yesterday's solar output to say much about today's demand directly — my best guess is that it's acting as an indirect proxy for season and weather, not that solar generation itself drives consumption. I'd want more than a SHAP ranking alone before treating this as more than a proxy effect.

8 Discussion: what this project can't do, and what a Vietnam version would need

- **This dataset is live and not fully reproducible.** Every number above reflects one snapshot; a fresh download will add rows without changing historical ones, but the exact row counts and test-set contents quoted here are tied to this specific download.
- **The 2024-2025 outage gaps are patched with a rough approximation.** An 83-hour linear interpolation can't recover real demand swings for that stretch — evaluation numbers touching that period are measured on partly reconstructed, not fully real, data.
- **No weather data** — the sharp winter-heating seasonality is only seen through the calendar, never an actual temperature reading.
- **Different climate and market structure than Vietnam.** Romania's winter-heating-dominant demand shape is close to the opposite of Vietnam's likely summer-cooling-dominant shape. The *methodology* transfers directly; the specific calendar, climate-driven seasonality, and holiday structure would not.
- **The renewable-share angle connects most directly to Vietnam's own situation:** Vietnam's grid is going through its own fast solar and wind buildout, with well-documented grid-stability and curtailment challenges. A genuine Vietnam version of this analysis would need Vietnam's own hourly generation-mix data, which I could not find in public, granular form.

9 Conclusion

Switching from a frozen US demand series to this live Romanian consumption-and-production dataset paid off in ways beyond just having more columns: it forced a genuine leakage-trap decision (lagging every generation-mix feature), surfaced a real, previously-unseen data-quality problem (multi-hour 2024-2025 outage gaps, structurally different from the DST issue), and gave two honest negative/surprising results worth reporting on their own terms — the seasonal-decomposition feature added nothing once properly leakage-checked, and yesterday's solar output ranks surprisingly high in the final model's own explanation of itself, for reasons I can guess at but haven't proven. LightGBM's win over both classical regression and a neural-network baseline held up on this dataset exactly as it did on the original US version — a second, independent piece of evidence that the highest common instinct ("try a neural net") isn't automatically the right one for a tabular, well-feature-engineered forecasting problem like this.